ARTIFICIAL INTELLIGENCE

Comprehensive Study Notes

Fundamentals • Machine Learning • Generative AI • LLMs • RAG • AI Agents

==========

1. WHAT IS ARTIFICIAL INTELLIGENCE?

Artificial Intelligence (AI) is a branch of computer science that builds systems capable of performing tasks that normally require human intelligence.

Examples:

- Understanding and generating language
- Recognizing images and speech
- Making predictions
- Solving problems
- Planning actions and making recommendations

AI systems do not necessarily think or understand in the same way humans do. Their capabilities depend on their design, training data, and available tools.

2. MAIN GOALS OF AI

- Perception: interpret images, sound, and other inputs.
- Reasoning: use information and rules to reach conclusions.
- Learning: improve performance from data or experience.
- Planning: select actions to achieve a goal.
- Natural language processing: work with human language.
- Robotics: sense and act in the physical world.

3. TYPES OF AI

A. By capability

- Narrow AI (Weak AI): designed for particular tasks. Most current AI systems fall into this category.
- General AI (AGI): a hypothetical system with broad, flexible abilities comparable to human intelligence across many tasks.
- Superintelligent AI: a hypothetical AI that surpasses human abilities across most intellectual domains.

B. By behavior

- Reactive systems: respond to current inputs without using a stored history in the way learning systems do.
- Limited-memory systems: use learned patterns or past information for a task.
- More advanced categories sometimes discussed in theory include theory-of-mind and self-aware AI; these are not established capabilities of current AI.

4. AI, MACHINE LEARNING, AND DEEP LEARNING

Artificial Intelligence (AI)

The broad field of building systems that perform intelligent tasks.

Machine Learning (ML)

A subset of AI in which systems learn patterns from data instead of relying only on explicitly written rules.

Deep Learning (DL)

A subset of ML that uses neural networks with many layers to learn complex patterns.

Relationship:

AI

■■■ Machine Learning

■■■ Deep Learning

Example:

- AI: a system that identifies spam.
- ML: the system learns patterns from labeled email examples.
- DL: a deep neural network learns complex representations from large datasets.

5. COMMON MACHINE-LEARNING APPROACHES

- Supervised learning: learns from labeled examples.

Example: predicting house prices from examples with known prices.

- Unsupervised learning: discovers structure in data without target labels.

Example: grouping customers by behavior.

- Semi-supervised learning: uses a small labeled dataset with a larger unlabeled dataset.
- Reinforcement learning: an agent learns actions through rewards and penalties.

Example: learning a strategy in a simulated game.

6. NEURAL NETWORKS

A neural network is a machine-learning model made of connected computational units, often organized into layers.

Typical layers:

- Input layer: receives data.
- Hidden layers: transform information and learn patterns.
- Output layer: produces a prediction or result.

Training adjusts model parameters to reduce a loss function. An optimizer, such as stochastic gradient descent or Adam, updates the parameters.

Important terms:

- Parameter: a learned value, such as a weight or bias.
- Loss function: measures how far a prediction is from the target.
- Epoch: one pass through the training dataset.
- Batch: a subset of examples processed together.
- Overfitting: performs well on training data but poorly on new data.

- Generalization: performance on unseen examples.
- Regularization: techniques that help reduce overfitting.

7. NATURAL LANGUAGE PROCESSING (NLP)

NLP enables computers to process and generate human language.

Applications:

- Translation
- Sentiment analysis
- Text classification
- Question answering
- Summarization
- Chatbots
- Information extraction

Tokenization:

Text is split into units called tokens. A token may be a word, part of a word, character, or another unit, depending on the tokenizer.

Embeddings:

Embeddings represent tokens, sentences, or other data as numeric vectors. Vectors with similar meanings or patterns may be close in the embedding space.

8. GENERATIVE AI

Generative AI creates new content based on patterns learned during training.

It can generate text, images, audio, video, and code.

Examples of uses:

- Drafting and summarizing documents
- Generating images from text
- Creating synthetic audio
- Helping write and explain code
- Extracting structured information from text

Generative AI can produce plausible-sounding but incorrect information. Its outputs should be checked when accuracy matters.

9. LARGE LANGUAGE MODELS (LLMs)

An LLM is a language model trained on large amounts of text and other data.

Many modern LLMs use the Transformer architecture.

Basic generation process:

1. Input text is tokenized.
2. Tokens are converted into numerical representations.
3. The model processes context and estimates likely next tokens.
4. Tokens are generated step by step.

5. The output tokens are converted back into text.

Key terms:

- Context window: the amount of input and conversation the model can consider at once, subject to the model's limits.
- Temperature: controls randomness in token selection. Lower values often give more predictable outputs; higher values can give more varied outputs.
- System instruction: high-level directions that guide the model's behavior.
- Prompt: the input or instructions provided to a model.
- Hallucination: an answer that is incorrect or unsupported but presented as if it were true.

10. PROMPT ENGINEERING

Prompt engineering means designing instructions and context to improve a model's output.

Useful practices:

- State the task clearly.
- Provide relevant context.
- Specify the desired format.
- Include examples when useful.
- Set constraints and explain what to avoid.
- Ask the model to identify uncertainty rather than invent facts.
- Validate the output, especially for important decisions.

Example:

"You are an information extraction assistant. Extract the movie title, year, and director from the paragraph. Return only valid JSON with keys title, year, and director. Use null when a field is not available."

11. STRUCTURED OUTPUT AND PYDANTIC

Structured output means asking a model to return data in a defined format, such as JSON, rather than unrestricted prose.

Benefits:

- Easier validation
- More reliable integration with software
- Predictable fields and data types

Pydantic is a Python library for defining data models and validating data.

A schema can specify required fields, types, and constraints. However, a schema does not guarantee that the model's extracted facts are correct; factual validation may still be needed.

12. EMBEDDINGS AND VECTOR SEARCH

An embedding model converts data such as text into vectors (lists of numbers) that represent useful semantic or other relationships.

Vector search retrieves vectors that are close to a query vector according to a similarity or distance measure.

Common measures:

- Cosine similarity
- Dot product
- Euclidean distance

Typical semantic-search workflow:

1. Split documents into chunks.
2. Create an embedding for each chunk.
3. Store vectors and associated text in a vector database or index.
4. Embed the user's query.
5. Retrieve the most relevant chunks.
6. Use the retrieved content to answer the question.

13. RETRIEVAL-AUGMENTED GENERATION (RAG)

RAG combines information retrieval with text generation. It supplies relevant external information to a language model so that the answer can be grounded in retrieved material.

Typical RAG pipeline:

1. Collect documents.
2. Clean and split documents into chunks.
3. Generate embeddings for the chunks.
4. Store chunks and vectors in an index or vector database.
5. Receive a user question.
6. Retrieve relevant chunks.
7. Optionally rerank or filter the retrieved chunks.
8. Place the question and retrieved context into a prompt.
9. Generate an answer.
10. Provide citations or references when available and verify the answer.

Important RAG concepts:

- Chunk size: amount of text stored in each chunk.
- Chunk overlap: shared text between adjacent chunks to preserve context.
- Retrieval quality: whether the correct passages are found.
- Reranking: reordering retrieved passages using a more precise relevance model.
- Grounding: basing the answer on supplied evidence.

- Evaluation: measuring retrieval quality, answer relevance, and factual support.

RAG can reduce unsupported answers but does not eliminate hallucinations.

It also depends on document quality, retrieval settings, and prompt design.

14. FINE-TUNING VS RAG

Fine-tuning:

- Further trains a model on examples for a desired behavior, style, or task.
- Changes model parameters.
- Useful when a model needs consistent task-specific behavior.
- Does not automatically provide up-to-date facts.

RAG:

- Retrieves external information at query time.
- Usually does not change model parameters.
- Useful when answers must use private, changing, or document-specific data.
- Depends on the quality of retrieval and source documents.

The two approaches can be combined when appropriate.

15. AI FRAMEWORKS AND TOOLS

- Python: widely used language for AI development.
- PyTorch / TensorFlow: libraries for building and training neural networks.
- Hugging Face: ecosystem for models, datasets, tokenizers, and ML tools.
- Ollama: tool for running supported language models locally.
- LangChain: framework for composing model calls, prompts, retrievers, tools,

and other application components.

- Vector databases: store and retrieve vectors and related metadata.
- APIs: allow applications to send requests to hosted models.

Choose tools based on the project requirements, hardware, privacy, cost, and deployment needs. Frameworks do not replace understanding the underlying concepts.

16. LOCAL MODELS VS API MODELS

Local model:

- Runs on your own computer or server.
- Can offer more control over data and offline use, depending on setup.
- Limited by available RAM, VRAM, storage, and model speed.
- Requires installation, downloads, and local resource management.

API model:

- Runs on a provider's infrastructure and is accessed over a network.
- Often needs less local hardware.
- May have usage costs, rate limits, and data-handling terms to consider.

- Depends on provider availability and network connectivity.

Before choosing, check licensing, privacy terms, resource requirements, and whether the model is suitable for the task.

17. AI AGENTS

An AI agent is a system that uses a model to pursue a goal through a sequence of steps. Depending on its design, it may plan, use tools, observe results, and adjust its next action.

Common components:

- Model: interprets context and selects actions.
- Tools: perform tasks such as searching, calculations, or database queries.
- State or memory: stores relevant information across steps.
- Planning and control logic: decides what to do next.
- Guardrails: restrict unsafe or unauthorized actions.

A chatbot that only responds to messages is not necessarily an agent. An agent typically has a workflow involving actions or tool use.

18. COMPUTER VISION

Computer vision enables systems to analyze images and video.

Tasks include:

- Image classification: assign a label to an image.
- Object detection: identify objects and their locations.
- Segmentation: classify pixels or image regions.
- OCR: extract text from images.
- Face recognition: identify or compare faces, with significant privacy and fairness considerations.

19. ROBOTICS AND SPEECH AI

Robotics combines perception, planning, and control to act in the physical world. Speech AI includes speech recognition (audio to text), speech synthesis (text to audio), and spoken-language understanding.

Examples include industrial robots, navigation systems, voice assistants, and accessibility tools.

20. EVALUATING AI SYSTEMS

AI should be evaluated using tests that match its intended use.

Possible measures:

- Accuracy: fraction of predictions that are correct.
- Precision: among predicted positives, the fraction that are truly positive.
- Recall: among actual positives, the fraction identified.
- F1 score: harmonic mean of precision and recall.

- Latency: time taken to return a result.
- Cost: compute or service expense.
- Robustness: performance under varied or imperfect inputs.
- For RAG: retrieval relevance, faithfulness to retrieved context, and answer usefulness.

No single metric is suitable for every task. Test on representative data and include edge cases.

21. LIMITATIONS, RISKS, AND RESPONSIBLE AI

- Bias: training data or system design can produce unfair outcomes.
- Privacy: sensitive information may be exposed or mishandled.
- Security: systems can be attacked or manipulated by malicious inputs.
- Hallucination: generated content can be false.
- Copyright and licensing: training data and outputs may raise legal issues.
- Explainability: some models are difficult to interpret.
- Environmental and financial costs: training and inference require resources.
- Overreliance: people may trust outputs without verification.

Good practices:

- Protect sensitive data.
- Verify important claims against reliable sources.
- Test for bias and failure cases.
- Apply access controls and logging where appropriate.
- Keep humans involved in high-impact decisions.
- Follow applicable laws, policies, and model licenses.

22. PRACTICAL AI PROJECT IDEAS

Beginner:

- Spam classifier
- Sentiment analysis tool
- FAQ chatbot
- Image classifier

Intermediate:

- PDF question-answering app using RAG
- Resume information extractor with structured output
- Semantic search for notes
- Meeting transcript summarizer

Advanced:

- RAG system with reranking and source citations
- Domain-specific evaluation pipeline
- Tool-using agent with permissions and logging

- Multimodal app that works with text and images

23. QUICK REVISION

AI: the broad field of systems performing intelligent tasks.

ML: systems learn patterns from data.

Deep learning: ML using multi-layer neural networks.

NLP: processing and generating human language.

Generative AI: creating new content.

LLM: a large language model that generates or processes language.

Token: a unit of text processed by a model.

Embedding: a numeric vector representation.

Vector search: retrieval based on vector similarity or distance.

Prompt engineering: designing instructions and context for model outputs.

RAG: retrieve relevant external information and use it to generate an answer.

Fine-tuning: further training a model for behavior or task adaptation.

Agent: a system that can use a model and tools to take goal-directed steps.

Hallucination: generated content that is false or unsupported.

Inference: using a trained model to produce a result.

Training: adjusting model parameters using data.

==========